

Modeling, Prototyping, Identification, and Control of a 4-DOF Robot Manipulator

System and Control Engineering Laboratory
Mechanical Engineering Department
Institut Teknologi Sepuluh Nopember

Abstract

This guideline defines an integrated undergraduate final project for a four-student team involving the design, construction, modeling, identification, simulation, control, and experimental validation of a four-degree-of-freedom robot manipulator. The project follows an iterative model-based engineering workflow rather than treating mechanical design, modeling, simulation, control, and testing as isolated activities. The minimum technical scope includes analytical kinematics and dynamics, CAD design, actuator and sensor selection, prototype fabrication, experimental parameter identification, MATLAB/Simulink and Simscape Multibody simulation, embedded control, and quantitative experimental validation.

Contents

1	Introduction	3
2	Project Overview	4
3	Project Objectives	5
3.1	Main Objective	5
3.2	Specific Objectives	5
4	Final Project Stages	5
4.1	Stage 1: Requirements and System Architecture	5
4.2	Stage 2: Mathematical Modeling	6
4.2.1	Kinematic Modeling	6
4.2.2	Workspace and Differential Kinematics	6
4.2.3	Dynamic Modeling	6
4.2.4	Static Verification	7
4.3	Stage 3: Mechanical and Mechatronic Design	7
4.3.1	Mechanical Design	7
4.3.2	Actuator Selection	7
4.3.3	Sensors and Electronics	8
4.3.4	CAD-to-Simscape Preparation	8
4.4	Stage 4: Prototype Manufacturing and Assembly	8
4.5	Stage 5: Parameter Determination and Identification	9
4.5.1	Nominal Parameters	9
4.5.2	Encoder and Torque Calibration	9
4.5.3	Friction and Gravity Identification	9
4.5.4	Dynamic Parameter Identification	9
4.5.5	Data Processing and Validation	10
4.6	Stage 6: Simscape Multibody Simulation	10
4.6.1	Model Development Levels	10
4.6.2	Analytical-to-Simscape Verification	11
4.6.3	Required Simulation Scenarios	11
4.7	Stage 7: Controller Design	11
4.7.1	Architecture and Trajectory Generation	11
4.7.2	Baseline and Model-Based Controllers	11
4.7.3	Required Comparison	12
4.8	Stage 8: Embedded Implementation	12
4.9	Stage 9: Experimental Validation	13
5	Team Organization	14
5.1	Student 1: Modeling and Kinematics Lead	14
5.2	Student 2: Mechanical Design and Prototype Lead	14
5.3	Student 3: Identification and Simulation Lead	14
5.4	Student 4: Control and Embedded Systems Lead	14
5.5	Shared Responsibilities	14
6	Expected Final Report Structure	14

1 Introduction

A robot manipulator is a programmable mechanical system designed to position and orient an end effector within a specified workspace. Its mechanical structure generally consists of a base, a sequence of rigid links connected by revolute or prismatic joints, actuators, sensors, a control unit, and an end effector. The end effector may be a gripper, welding torch, machining tool, camera, or another device selected for the intended task. By coordinating the motion of its joints, the manipulator can produce controlled arm-like motion and interact physically with objects and the surrounding environment [1, 2].

The number of independent joint variables is known as the robot's degrees of freedom (DOF). A 4-DOF robot manipulator has four independently controlled joint motions. Depending on its mechanical configuration, these motions may control the end-effector position and part of its orientation. Compared with a six-degree-of-freedom industrial robot, a 4-DOF manipulator has a more limited set of possible end-effector poses. However, its lower mechanical complexity, computational requirement, and cost make it appropriate for education, laboratory research, light automation, and applications in which full spatial orientation is not required.

Robot manipulators are used to improve productivity, repeatability, precision, safety, and consistency. They are particularly valuable for tasks that are repetitive, physically demanding, hazardous, or difficult to perform accurately by human operators. Their programmable motion and interchangeable end effectors enable one robotic platform to be adapted to different applications. Robot manipulators are used in manufacturing, logistics, inspection, rehabilitation, service systems, and research [3, 1].

Although a robot manipulator may appear to perform a simple motion, achieving accurate, fast, smooth, energy-efficient, and safe operation requires several interconnected engineering problems to be solved.

1. **Kinematic modeling and workspace analysis.** The relationship between joint variables and end-effector position and orientation must be formulated accurately. Forward kinematics determines the end-effector pose from the joint variables, whereas inverse kinematics determines suitable joint variables for a desired pose. Multiple solutions, unreachable targets, joint limits, and singular configurations must be considered.
2. **Dynamic modeling.** The model must account for link and joint inertia, gravity, Coriolis and centrifugal effects, friction, actuator dynamics, and payload forces. Robot dynamics are commonly represented by differential equations derived using Euler-Lagrange methods. Accurate dynamic models are important for simulation, actuator selection, torque prediction, and model-based controller design [4].
3. **Parameter uncertainty and system identification.** Actual link masses, centers of mass, moments of inertia, motor constants, friction coefficients, and gearbox efficiencies may differ from nominal design values. Payload properties may also change during operation. Experimental identification and parameter-estimation methods are therefore required to obtain a model that adequately represents the physical robot.
4. **Trajectory planning.** Joint-space or Cartesian-space trajectories must satisfy position, velocity, acceleration, jerk, torque, and workspace constraints. The motion should avoid obstacles and singularities while meeting requirements for cycle time, smoothness, accuracy, and energy consumption. Practical trajectory planning should also account for robot dynamics.

5. **Feedback control and dynamic coupling.** Each actuator must track its reference despite nonlinear dynamics and interactions among the joints. Motion at one joint can create forces and torques at other joints. Suitable control methods may include PID, computed-torque, robust, adaptive, sliding-mode, or impedance control.
6. **Friction, backlash, compliance, and vibration.** Gear trains and joints introduce Coulomb friction, viscous friction, dead zones, and backlash. Lightweight links may bend and vibrate during rapid acceleration or stopping. These effects reduce positioning accuracy and may cause oscillation, particularly during slow motion or motion reversal. Elastic behavior and vibration therefore need to be modeled, measured, and compensated.
7. **Sensing and state estimation.** Encoders, current sensors, force sensors, cameras, and inertial sensors are affected by noise, bias, delay, limited resolution, and sampling constraints. Reliable estimates of position, velocity, acceleration, torque, and external disturbances are needed for closed-loop control and condition monitoring.
8. **Actuator selection and physical constraints.** Motors and transmissions must provide the required torque, speed, resolution, and thermal capacity. The design must respect actuator saturation, power limitations, joint travel, structural strength, and allowable deflection. An unsuitable actuator or gear ratio can prevent the control system from achieving the desired performance even when the control algorithm is theoretically correct.
9. **Robustness to payload variation and disturbances.** Manipulator dynamics are nonlinear and are affected by changing loads, modeling errors, and external disturbances. Maintaining stable and accurate performance under these uncertainties is a central engineering objective in practical robot control [3].
10. **Safety and experimental validation.** Unexpected motion, software errors, collisions, and excessive joint torque can damage the robot or its surroundings. Mechanical stops, current and torque limits, emergency-stop functions, conservative test speeds, and staged validation from simulation to hardware are essential. The controller should be evaluated quantitatively under nominal and uncertain operating conditions before higher-speed operation is attempted.

These challenges are strongly coupled. For example, inaccurate dynamic parameters can degrade computed-torque control, friction can interfere with collision detection, and aggressive trajectory planning can excite structural vibration or saturate an actuator. Consequently, a well-scoped undergraduate final project should select one principal engineering challenge, establish a baseline method, develop one improved approach, and validate the improvement using clearly defined performance measures.

2 Project Overview

The project involves designing and building a four-degree-of-freedom serial robot manipulator, developing its mathematical and simulation models, identifying its physical parameters experimentally, designing joint-space controllers, implementing the controllers on an embedded platform, and validating the complete system experimentally.

The recommended manipulator configuration is:

- Joint 1: base rotation

- Joint 2: shoulder rotation
- Joint 3: elbow rotation
- Joint 4: wrist rotation
- End effector: simple gripper, treated as an auxiliary actuator rather than a fifth manipulator DOF
- Joint type: four revolute joints, or RRRR configuration
- Application: tabletop pick-and-place or path-tracking operation
- Payload: approximately 0.1 to 0.5 kg, depending on actuator capability
- Control objective: joint-space trajectory tracking
- Baseline controller: independent-joint PID or PD control
- Proposed advanced controller: computed-torque control with friction or payload compensation

3 Project Objectives

3.1 Main Objective

To develop and experimentally validate an integrated 4-DOF robot manipulator through mathematical modeling, mechanical and mechatronic design, dynamic parameter identification, multi-body simulation, controller design, and embedded implementation.

3.2 Specific Objectives

At the completion of the project, the team should be able to:

1. Establish functional and engineering requirements for a 4-DOF manipulator.
2. Design its mechanical structure, actuators, sensors, electronics, and end effector.
3. Develop forward and inverse kinematic models.
4. Build a CAD model and physical prototype.
5. Estimate important physical parameters experimentally.
6. Create and calibrate a Simscape Multibody digital model.
7. Design and compare baseline and model-based controllers.
8. Implement real-time control on an embedded platform.

4 Final Project Stages

4.1 Stage 1: Requirements and System Architecture

The team should define what the robot must do before developing mathematical models or selecting motors.

The output of this stage are:

- System requirements specification
- Preliminary Quantitative Targets
- Preliminary robot configuration
- Project schedule
- Design-review presentation

4.2 Stage 2: Mathematical Modeling**4.2.1 Kinematic Modeling**

The team shall define coordinate frames, construct a Denavit–Hartenberg table, and derive

$${}^{i-1}T_i(q_i), \quad {}^0T_4(q) = {}^0T_1{}^1T_2{}^2T_3{}^3T_4$$

The model shall provide the end-effector position and relevant orientation variables. An inverse-kinematic procedure shall be developed, checked against joint limits, and verified using CAD and numerical results. Joint zero positions must be defined using mechanical references, encoder offsets, or a documented calibration pose.

4.2.2 Workspace and Differential Kinematics

The analysis shall include maximum and minimum reach, reachable workspace, self-collision and table-interference regions, joint-limit effects, and singular configurations. The manipulator Jacobian shall satisfy

$$\dot{\mathbf{x}} = \mathbf{J}(\mathbf{q})\dot{\mathbf{q}}$$

The team must define the four task variables used by the 4-DOF arrangement, for example

$$\mathbf{x} = \begin{bmatrix} x & y & z & \phi \end{bmatrix}^T$$

Jacobian rank, condition number, or another manipulability measure should be evaluated.

4.2.3 Dynamic Modeling

The joint-space model shall be written as

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{G}(\mathbf{q}) + \mathbf{F}(\dot{\mathbf{q}}) = \boldsymbol{\tau} \quad (1)$$

The recommended derivation uses the Euler–Lagrange equation

$$\frac{d}{dt} \left(\frac{\partial L}{\partial \dot{q}_i} \right) - \frac{\partial L}{\partial q_i} = \tau_i, \quad L = K - P \quad (2)$$

where K is total kinetic energy and P is total potential energy.

A practical friction model is

$$F_i(\dot{q}_i) = F_{c,i} \operatorname{sgn}(\dot{q}_i) + b_i \dot{q}_i \quad (3)$$

The primary model may assume rigid links, fixed ideal axes, negligible gearbox elasticity, and negligible motor electrical dynamics for sufficiently slow motion. Each assumption must be documented and reconsidered if experiments reveal an important neglected effect.

4.2.4 Static Verification

Before dynamic simulation, verify the gravity model at several poses using

$$\boldsymbol{\tau}_{\text{static}} = G(\mathbf{q}).$$

This check detects sign and frame errors and supports actuator sizing.

The output of this stage are:

- Coordinate-frame diagram and DH table
- Forward and inverse kinematic models
- Workspace and singularity plots
- Jacobian analysis
- Dynamic equations and assumptions
- Static torque calculations
- MATLAB verification scripts

4.3 Stage 3: Mechanical and Mechatronic Design

4.3.1 Mechanical Design

The CAD assembly shall include a base, four links, shafts, bearings, motor mounts, transmissions, encoder mounts, mechanical stops, cable routing, and end-effector interface. The design should permit inspection, calibration, and replacement of joints and sensors without complete disassembly.

Link dimensions shall be based on workspace, payload, torque, stiffness, mass distribution, manufacturability, cost, and available facilities. Long links should not be selected before the shoulder and base torque requirements are evaluated.

4.3.2 Actuator Selection

For each joint, estimate gravitational and acceleration torque:

$$\tau_{g,i} = \sum_j m_j g r_{ij}, \quad \tau_{a,i} \approx J_{\text{eq},i} \ddot{q}_i.$$

The preliminary required torque is

$$\tau_{\text{req},i} = SF (\tau_{g,i} + \tau_{a,i} + \tau_{f,i}), \quad (4)$$

where SF is a justified safety factor, initially about 1.5–2. Continuous torque, peak torque, speed, thermal limitations, gear ratio, backlash, backdrivability, encoder availability, driver compatibility, power, and cost shall be considered separately.

4.3.3 Sensors and Electronics

Each joint requires position feedback. Identify suitable position and velocity sensor. Possible choices include incremental or absolute encoders, optical or magnetic sensors, and integrated servo feedback. Current sensing is recommended for torque estimation and protection.

The electronics shall include an embedded controller, four motor drivers, encoder interfaces, protected power distribution, emergency stop, limit inputs, communication, and data logging. The emergency stop must disable actuator power through hardware rather than software alone.

4.3.4 CAD-to-Simscape Preparation

The CAD model shall have clean joints, consistent frames and units, mass properties, and unambiguous link names. MATLAB's `smimport` can reconstruct bodies, constraints, and joints from suitable CAD XML, URDF, or `rigidBodyTree` sources. Supported CAD export workflows use Simscape Multibody Link or Onshape conversion tools [7, 8, 9]. Joint limits from CAD imports may require manual entry in the Simscape joint blocks [8].

The output of this stage are:

- Complete CAD assembly and manufacturing drawings
- Bill of materials
- Motor, transmission, and structural calculations
- Sensor and controller selection justification
- Electrical block and wiring diagrams
- Power budget and safety design

4.4 Stage 4: Prototype Manufacturing and Assembly

For first prototype, the team may use 3D-printing at laboratory to print the linkage frames. Load-bearing printed parts require explicit structural justification. Before power is applied, inspect joint alignment, bearings, shafts, range of motion, backlash, cable clearance, mechanical stops, fasteners, mass distribution, encoders, and emergency-stop continuity.

The progressive power-up sequence is:

1. controller without motor power;
2. sensor reading only;
3. one motor with conservative current or voltage limits;
4. individual-joint open-loop tests;
5. individual-joint closed-loop tests;
6. multiple joints without payload;
7. low-speed complete manipulator motion; and
8. motion with nominal payload.

The output of this stage are:

- Physical prototype and assembly procedure
- Inspection and power-up checklists
- Measured joint limits and backlash
- Initial calibration results
- Safety-test record

4.5 Stage 5: Parameter Determination and Identification**4.5.1 Nominal Parameters**

Initial parameters shall be obtained from CAD mass properties, datasheets, direct measurement, pendulum tests, motor tests, gearbox information, and static balancing. Required parameters include link masses, centers of mass, inertia estimates, gear ratios, torque constants, encoder scales, friction, joint offsets, payload mass, and payload center of mass.

4.5.2 Encoder and Torque Calibration

For an incremental encoder,

$$q_i = \frac{2\pi}{N_i n_i} (c_i - c_{0,i}), \quad (5)$$

where N_i is counts per motor revolution, n_i is gear ratio, and $c_{0,i}$ is the zero count.

Motor and approximate joint torque may be estimated as

$$\tau_{m,i} = K_{t,i} I_i, \quad \tau_i = \eta_i n_i K_{t,i} I_i.$$

The effective torque constant should be checked using a known lever arm and calibrated weights.

4.5.3 Friction and Gravity Identification

Conduct constant positive and negative velocity tests, several speed levels, low-speed motion, and reversals. Identify at least Coulomb and viscous friction. At static equilibrium,

$$\tau_i \approx G_i(\mathbf{q}) + \tau_{f,i},$$

which supports validation of gravity parameters and current-to-torque scaling.

4.5.4 Dynamic Parameter Identification

Express the dynamics in regression form:

$$\boldsymbol{\tau} = \mathbf{Y}(\mathbf{q}, \dot{\mathbf{q}}, \ddot{\mathbf{q}}) \boldsymbol{\theta} + \boldsymbol{\varepsilon}. \quad (6)$$

A least-squares estimate is conceptually

$$\hat{\boldsymbol{\theta}} = (\mathbf{Y}^T \mathbf{Y})^{-1} \mathbf{Y}^T \boldsymbol{\tau}, \quad (7)$$

but QR decomposition, singular-value decomposition, or a pseudoinverse should be used numerically.

A bounded multi-sine excitation trajectory may be used:

$$q_i(t) = q_{i,0} + \sum_{k=1}^{n_h} [a_{ik} \sin(k\omega t) + b_{ik} \cos(k\omega t)]. \quad (8)$$

It must respect position, velocity, acceleration, torque, collision, and workspace limits. Informative excitation and good conditioning of the observation matrix are important for reliable identification [10, 11].

4.5.5 Data Processing and Validation

The team shall document sampling frequency, synchronization, filtering, derivative estimation, current-to-torque conversion, outlier handling, and the separation of identification and validation data. Velocity and acceleration should be obtained through justified filtered differentiation or local polynomial estimation.

Validation must use an independent trajectory and should include multiple payload conditions. A possible normalized model-fit metric is

$$\text{NRMSE} = 1 - \frac{\|\tau_{\text{measured}} - \tau_{\text{predicted}}\|}{\|\tau_{\text{measured}} - \bar{\tau}_{\text{measured}}\|}. \quad (9)$$

The output of this stage are:

- Identification and calibration plan
- Raw and processed datasets
- Identified parameter table with uncertainty information
- Measured-versus-predicted torque plots
- Independent model-validation results

4.6 Stage 6: Simscape Multibody Simulation

4.6.1 Model Development Levels

1. **Ideal rigid-body model:** links, joints, gravity, limits, ideal torque inputs, and sensors.
2. **Nominal actuator model:** motor inertia, gear ratio, torque constant, saturation, preliminary friction, and encoder effects.
3. **Calibrated model:** measured masses, identified centers of mass, friction, torque constants, limits, and payload.
4. **Control-oriented model:** trajectory generator, controller, sampling, quantization, noise, delay, and saturation.

After import, verify joint types and axes, coordinate systems, masses, inertias, gravity, initial pose, limits, units, transforms, and solver settings. A model is not considered verified merely because its animation appears correct.

4.6.2 Analytical-to-Simscape Verification

Compare analytical and Simscape results for static gravity torque, single-joint motion, slow multi-joint motion, dynamic trajectories, and payload variation. Investigate differences caused by frame definitions, inertia conventions, gear ratios, friction, actuator dynamics, or solver settings.

4.6.3 Required Simulation Scenarios

- Step response for each joint
- Point-to-point and continuous trajectories
- Nominal and increased payloads
- Parameter mismatch
- Measurement noise
- Torque saturation
- External disturbance

The output of this stage are:

- Simscape Multibody model and parameter file
- CAD import documentation
- Analytical-versus-Simscape verification
- Simulation scenario library

4.7 Stage 7: Controller Design

4.7.1 Architecture and Trajectory Generation

A cascaded architecture is recommended, with an outer joint-position loop and optional inner velocity or current loop. It shall include a trajectory generator, safety supervisor, and data logger.

Cubic and quintic trajectories should be investigated. A quintic joint reference is

$$q_d(t) = a_0 + a_1 t + a_2 t^2 + a_3 t^3 + a_4 t^4 + a_5 t^5,$$

subject to position, velocity, acceleration, and torque constraints.

4.7.2 Baseline and Model-Based Controllers

The baseline controller is

$$\tau = K_p(q_d - q) + K_d(\dot{q}_d - \dot{q}), \quad (10)$$

with optional integral action, output saturation, anti-windup, derivative filtering, joint-limit protection, and reference-rate limiting.

An intermediate gravity-compensated controller is

$$\tau = K_p e + K_d \dot{e} + \hat{G}(q). \quad (11)$$

The recommended advanced controller is

$$\boldsymbol{\tau} = \hat{\mathbf{M}}(\mathbf{q})\mathbf{v} + \hat{\mathbf{C}}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \hat{\mathbf{G}}(\mathbf{q}) + \hat{\mathbf{F}}(\dot{\mathbf{q}}), \quad (12)$$

where

$$\mathbf{v} = \ddot{\mathbf{q}}_d + K_d(\dot{\mathbf{q}}_d - \dot{\mathbf{q}}) + K_p(\mathbf{q}_d - \mathbf{q}). \quad (13)$$

With a sufficiently accurate model, the approximate error dynamics are

$$\ddot{\mathbf{e}} + K_d\dot{\mathbf{e}} + K_p\mathbf{e} = 0.$$

4.7.3 Required Comparison

Compare independent-joint control, gravity-compensated control, and computed-torque control using nominal and increased payloads, moderate- and higher-acceleration trajectories, and a safe disturbance test. Evaluation shall include error, torque, current, overshoot, energy, computation, temperature, and robustness.

The output of this stage are:

- Controller derivations and block diagrams
- Gain-selection and tuning procedure
- Justification of the selected final controller

4.8 Stage 8: Embedded Implementation

The embedded platform shall provide four encoder interfaces, adequate motor-control outputs, deterministic sampling, timer interrupts, communication, emergency-stop input, and data transfer.

Recommended modules are startup and self-test, sensor acquisition, encoder conversion, velocity estimation, trajectory generation, control law, actuation, saturation, safety supervision, communication, logging, and fault handling.

At each control cycle, the software shall read sensors, estimate state, update the reference, calculate control action, apply limits, command the drivers, log variables, and check safety conditions.

The team shall measure actual sample period, jitter, worst-case computation time, communication delay, and missed cycles.

The system shall stop when the emergency stop is pressed, joint limits are exceeded, sensors become invalid, current or error exceeds a threshold, communication is lost, or real-time execution fails.

The output of this stage are:

- Embedded source code and architecture diagram
- Timing and jitter analysis
- Communication protocol and calibration file
- Safety-state diagram

- Single-joint and multi-joint commissioning results

4.9 Stage 9: Experimental Validation

Validation shall determine whether the model represents the robot, whether the controller tracks the desired motion, whether model-based control outperforms the baseline, and whether the robot completes the practical task safely and repeatedly.

Required Experiments

1. **Kinematic validation:** compare predicted, simulated, and measured end-effector positions throughout the workspace.
2. **Static gravity validation:** compare analytical, Simscape, and current-derived torques at several poses and payloads.
3. **Single-joint tests:** step, ramp, smooth point-to-point, positive and negative motion, and different speeds.
4. **Multi-joint tracking:** point-to-point pick-and-place and continuous trajectories.
5. **Payload robustness:** no payload, nominal payload, and safely increased payload.
6. **Repeatability:** command the same target at least 20 times and calculate dispersion.

Kinematic position error is

$$e_p = \|\mathbf{p}_{\text{measured}} - \mathbf{p}_{\text{predicted}}\|.$$

Joint and Cartesian RMSE are

$$\text{RMSE}_i = \sqrt{\frac{1}{N} \sum_{k=1}^N (q_{d,i}(k) - q_i(k))^2}, \quad (14)$$

$$\text{RMSE}_p = \sqrt{\frac{1}{N} \sum_{k=1}^N \|\mathbf{p}_d(k) - \mathbf{p}(k)\|^2}. \quad (15)$$

Additional indices are

$$\text{IAE} = \int_0^T |e(t)| dt, \quad (16)$$

$$J_\tau = \int_0^T \boldsymbol{\tau}^T(t) \boldsymbol{\tau}(t) dt, \quad (17)$$

$$E_m = \int_0^T \sum_{i=1}^4 |\tau_i(t) \dot{q}_i(t)| dt. \quad (18)$$

Important experiments should be repeated at least five times. The report shall give means, standard deviations, identical initial conditions where possible, controller gains, supply voltage, and access to raw data.

The output of this stage are:

- Experimental and safety plans
- Raw and processed datasets

- Controller-comparison plots and statistics
- Model-validation results
- Demonstration video
- Discrepancy analysis and final validation report

5 Team Organization

All students shall understand the entire system, but each will lead one technical area.

5.1 Student 1: Modeling and Kinematics Lead

Lead coordinate frames, kinematics, Jacobian, singularity analysis, dynamics, analytical MATLAB model, and analytical-to-Simscape comparison. Support computed-torque implementation and report integration.

5.2 Student 2: Mechanical Design and Prototype Lead

Lead CAD, structure, actuator and transmission selection, manufacturing drawings, assembly, mechanical calibration, and CAD-to-Simscape preparation. Support mass-property measurement and safety fixtures.

5.3 Student 3: Identification and Simulation Lead

Lead Simscape modeling, actuator and sensor models, calibration experiments, data processing, friction and dynamic parameter identification, and model calibration. Maintain the experimental dataset repository.

5.4 Student 4: Control and Embedded Systems Lead

Lead baseline, gravity-compensated, and computed-torque control, embedded programming, driver and sensor interfaces, timing analysis, and safety supervision. Coordinate experimental control testing.

5.5 Shared Responsibilities

All members shall participate in requirements, design reviews, risk assessment, integration, validation, interpretation, and demonstration. The project shall not become four unrelated subprojects.

6 Expected Final Report Structure

1. Introduction
2. Literature Review
3. Requirements and System Architecture
4. Robot Configuration and Mechanical Design
5. Kinematic Modeling

6. Dynamic Modeling
7. Actuator, Sensor, and Electronics Design
8. Prototype Manufacturing and Assembly
9. Parameter Calibration and Identification
10. Simscape Multibody Model
11. Controller Design
12. Embedded Implementation
13. Experimental Methodology
14. Results and Discussion
15. Model and Controller Comparison
16. Conclusions and Recommendations
17. References
18. Appendices: drawings, bill of materials, parameters, source code, risk assessment, calibration data, and datasets

References

- [1] S. Sharma, "What Are Manipulator Robots? Understanding Their Design, Types, and Applications," *Wevolver*, 2025. Available: <https://www.wevolver.com/article/robot-manipulator>
- [2] R. Tedrake, *Robotic Manipulation: Perception, Planning, and Control*, Massachusetts Institute of Technology, 2026. Available: <https://manipulation.csail.mit.edu/>
- [3] A. Ghorbanpour, "Cooperative Robot Manipulators Dynamical Modeling and Control: An Overview," *Dynamics*, vol. 3, no. 4, pp. 820–854, 2023. Available: <https://doi.org/10.3390/dynamics3040045>
- [4] J. Norris, "Dynamics: Modeling, Motion Planning, and Control of Manipulators and Mobile Robots," in *Introduction to Robotics*. Available: <https://opentextbooks.clemson.edu/wangrobotics/chapter/dynamics/>
- [5] MathWorks, "Simscape Multibody Documentation," Available: <https://www.mathworks.com/help/sm/index.html>.
- [6] MathWorks, "Industrial Robot Models in Simscape," GitHub repository. Available: <https://github.com/mathworks/Industrial-Robots-Simscape>.
- [7] MathWorks, "Model Import: Import a Multibody Model into the Simscape Multibody Environment," Available: <https://www.mathworks.com/help/sm/cad-import.html>.
- [8] MathWorks, "smimport: Import a CAD, URDF, or Robotics System Toolbox Model," Available: <https://www.mathworks.com/help/sm/ref/smimport.html>.
- [9] MathWorks, "Import a CAD Assembly Model," Available: <https://www.mathworks.com/help/sm/ug/import-robot-arm-model.html>.

- [10] H. Tian, M. Huber, C. E. Mower, Z. Han, C. Li, X. Duan, and C. Bergeles, "Excitation Trajectory Optimization for Dynamic Parameter Identification Using Virtual Constraints in Hands-on Robotic System," *arXiv preprint arXiv:2401.16566*, 2024. Available: <https://arxiv.org/abs/2401.16566>.
- [11] C. Li, H. Qian, Z. Li, and M. Li, "An Excitation Trajectory for Dynamic Parameter Identification of Robot Using Logistic Function and Depth-First Search," *Journal of Dynamic Systems, Measurement, and Control*, vol. 147, no. 4, 2025. Available: <https://doi.org/10.1115/1.4068042>.
- [12] Long Vo, "Manipulator-X Simulation: 4-DOF Robotic Arm Simulator," GitHub repository. Available: <https://github.com/longvotheengineer/Manipulator-X-simulation>
- [13] A. Johorey, "4-DOF Robotic Manipulator: Kinematics, Dynamics and Control," GitHub repository. Available: <https://github.com/Aditya-Johorey/4-joint-robot-manipulator-design-and-simulation>